

N3 Banco de Dados

Alunos: Gustavo Ariel Barbosa e Paulo Guedes

Engenharia de Software – 3A

Contexto da atividade:

Desenvolver uma aplicação servidora em que cada prestador de serviço: código_prestador, nome_prestador e tempo_experiencia, tenha uma categoria: id_categoria, nome_categoria principal, como carpintaria, eletricitista, encanador, O serviço id_servico, nome_servico e vlr_servico realizado pelo prestador tem um valor fixo de R\$ 80,00 à hora, mas esse valor recebe acréscimo de acordo com a experiência do prestador: se a experiência é de 3 anos, acrescentar 30%, se for maior que 3 anos e menor igual a 5 anos acrescentar 50%, se maior que 5 anos acrescentar 75%. Para a implementação da persistência de dados, utilize a técnica de ORM - Object Relational Mapping. Essa aplicação tem que atender as requisições CRUD oriundas de qualquer cliente-server por meio de uma API Rest. Como também, permitir consultas de prestador por categoria e por serviço principal. A tecnologia para a implementação da aplicação é de livre escolha pela dupla. Além disso, inserir a utilização de token (JWT) em um dos end-points da API ou se preferir implemente uma funcionalidade de login (usuário e senha) com token.

Na implementação do código foi utilizada a biblioteca 'sequelize', onde facilita a integração entre API e banco de dados, sendo assim, pode haver algumas leves diferenças entre o código implementado na API ORM e as consultas apresentadas aqui neste documento.

Comandos SQL

a. Database

```
CREATE DATABASE prestadores_db;  
USE prestadores_db;
```

b. Tabela Categoria

```
CREATE TABLE Categorias (  
  id_categoria int primary key auto_increment,  
  nome_categoria varchar(100) not null,  
  createdAt datetime,  
  updatedAt datetime  
);
```

c. Tabela Prestador

```
CREATE TABLE Prestadores (  
  codigo_prestador int primary key auto_increment,  
  nome_prestador varchar(150) not null,  
  tempo_experiencia int default 0,  
  id_categoria int,  
  createdAt datetime,  
  updatedAt datetime,  
  foreign key (id_categoria) references Categorias(id_categoria)  
);
```

d. Tabela Servico

```
CREATE TABLE Servicos (  
  id_servico int primary key auto_increment,  
  nome_servico varchar(150) not null,  
  vlr_servico decimal(5, 2) default 80.00,  
  codigo_prestador int,  
  createdAt datetime,  
  updatedAt datetime,  
  foreign key (codigo_prestador) references Prestadores(codigo_prestador)  
);
```

e. Inserindo valores nas tabelas para utilizar na API

```
INSERT INTO Categorias (nome_categoria) VALUES
```

```
('Carpintaria'), ('Eletricista'), ('Encanador'), ('Pintor'), ('Pedreiro');
```

```
INSERT INTO Prestadores (nome_prestador, tempo_experiencia, id_categoria)  
VALUES
```

```
('Joao Silva', 3, 1),  
( 'Pedro Alves', 2, 1),  
( 'Lucas Martins', 6, 1),  
( 'Maria Santos', 4, 2),  
( 'Ana Costa', 5, 2),  
( 'Carlos Eduardo', 8, 2),  
( 'Jose Oliveira', 3, 3),  
( 'Roberto Lima', 7, 3),  
( 'Fernando Souza', 4, 4),  
( 'Ricardo Gomes', 10, 4),  
( 'Antonio Dias', 3, 5),  
( 'Marcos Pereira', 6, 5);
```

```
INSERT INTO Servicos (nome_servico, codigo_prestador) VALUES
```

```
('Instalacao de Porta', 1),  
( 'Fabricacao de Moveel Planejado', 1),  
( 'Reparo em Janela', 2),  
( 'Construcao de Deck', 3),  
( 'Instalacao Eletrica Residencial', 4),  
( 'Manutencao de Quadro Eletrico', 4),  
( 'Instalacao de Ar Condicionado', 5),  
( 'Projeto Eletrico Industrial', 6),  
( 'Reparo de Vazamento', 7),  
( 'Instalacao de Sistema Hidraulico', 7),  
( 'Desentupimento', 8),  
( 'Instalacao de Aquecedor', 8),  
( 'Pintura Residencial', 9),  
( 'Pintura Comercial', 9),  
( 'Textura e Grafiato', 10),  
( 'Construcao de Muro', 11),  
( 'Assentamento de Piso', 11),  
( 'Reforma Estrutural', 12);
```

2. Comando SQL dos endpoints

a. Categorias (/api/categorias/...)

```
-- Listar categorias (GET .../)  
SELECT c.id_categoria, c.nome_categoria,  
       p.codigo_prestador,  
       p.nome_prestador,  
       p.tempo_experiencia  
FROM categorias c;  
  
-- Adicionar categoria (Na API ORM, o nome da categoria vem como parâmetro da  
função que chama o Sequelize para comunicar com o BD, se fosse feito com  
mySQL puro nós precisaríamos aplicar Procedure) (POST .../)  
INSERT INTO categorias (nome_categoria) VALUES ({NOME DA CATEGORIA});  
  
-- Buscar categoria por ID (GET .../:id)  
SELECT  
  c.id_categoria,  
  c.nome_categoria,  
  c.createdAt,  
  c.updatedAt,  
  p.codigo_prestador,  
  p.nome_prestador,  
  p.tempo_experiencia,  
  p.id_categoria AS prestador_id_categoria,  
  p.createdAt AS prestador_createdAt,  
  p.updatedAt AS prestador_updatedAt  
FROM categorias c  
INNER JOIN prestadores p ON c.id_categoria = p.id_categoria  
WHERE c.id_categoria = {ID DA CATEGORIA};  
  
-- Atualizar categoria (PUT .../:id)  
UPDATE categorias  
SET nome_categoria = '{NOVO NOME}'  
WHERE id_categoria = {ID DA CATEGORIA};  
  
-- Deletar categoria (Se houver prestadores vinculados, ocorrerá erro) (DELETE .../:id)  
DELETE FROM categorias WHERE id_categoria = {ID DA CATEGORIA};
```

b. Prestadores (/api/prestadores/...)

-- Listar prestadores (GET .../)

```
SELECT
  p.codigo_prestador, p.nome_prestador, p.tempo_experiencia,
  p.id_categoria, p.createdAt, p.updatedAt,
  c.id_categoria AS categoria_id, c.nome_categoria, s.id_servico,
  s.nome_servico, s.vlr_servico
FROM prestadores p
INNER JOIN categorias c ON p.id_categoria = c.id_categoria
INNER JOIN servicos s ON p.codigo_prestador = s.codigo_prestador
ORDER BY p.codigo_prestador;
```

-- Adicionar prestador (POST .../)

```
INSERT INTO prestadores (nome_prestador, tempo_experiencia, id_categoria)
VALUES ({NOME}, {TEMPO_EXP}, {ID_CATEGORIA});
```

-- Buscar prestador por ID (GET .../:id)

```
SELECT
  p.codigo_prestador, p.nome_prestador, p.tempo_experiencia,
  p.id_categoria, p.createdAt, p.updatedAt,
  c.id_categoria AS categoria_id, c.nome_categoria,
  c.createdAt AS categoria_createdAt,
  c.updatedAt AS categoria_updatedAt,
  s.id_servico, s.nome_servico, s.vlr_servico,
  s.codigo_prestador AS servico_codigo_prestador,
  s.createdAt AS servico_createdAt,
  s.updatedAt AS servico_updatedAt
FROM prestadores p
INNER JOIN categorias c ON p.id_categoria = c.id_categoria
INNER JOIN servicos s ON p.codigo_prestador = s.codigo_prestador
WHERE p.codigo_prestador = {ID DO PRESTADOR};
```

-- Buscar prestadores por categoria

```
SELECT
  p.codigo_prestador, p.nome_prestador, p.tempo_experiencia,
  p.id_categoria, p.createdAt, p.updatedAt,
  c.id_categoria AS categoria_id,
  c.nome_categoria, s.id_servico, s.nome_servico,
  s.vlr_servico
FROM prestadores p
INNER JOIN categorias c ON p.id_categoria = c.id_categoria
INNER JOIN servicos s ON p.codigo_prestador = s.codigo_prestador
WHERE p.id_categoria = {ID DA CATEGORIA}
ORDER BY p.nome_prestador;
```

```

-- Buscar prestadores por serviço
SELECT
  p.codigo_prestador, p.nome_prestador, p.tempo_experiencia,
  p.id_categoria, p.createdAt, p.updatedAt,
  s.id_servico, s.nome_servico, s.vlr_servico
FROM prestadores p
INNER JOIN servicos s ON p.codigo_prestador = s.codigo_prestador
WHERE s.id_servico = {ID DO SERVICIO}
ORDER BY p.nome_prestador;

-- Atualizar prestador
UPDATE prestadores
SET nome_prestador = {NOVO NOME},
    tempo_experiencia = {NOVO TEMPO},
    id_categoria = {NOVA CATEGORIA},
WHERE codigo_prestador = {ID DO PRESTADOR};

-- Deletar prestador (Pode dar erro se houver serviço vinculado)
DELETE FROM prestadores WHERE codigo_prestador = {ID DO PRESTADOR};

```

c. Serviços (/api/servicos/...)

```

-- Listar Serviços
SELECT
  s.id_servico, s.nome_servico, s.vlr_servico, s.codigo_prestador,
  s.createdAt, s.updatedAt, p.codigo_prestador AS prestador_codigo,
  p.nome_prestador, p.tempo_experiencia,
  p.id_categoria AS prestador_id_categoria, p.createdAt AS prestador_createdAt,
  p.updatedAt AS prestador_updatedAt, c.id_categoria, c.nome_categoria,
  c.createdAt AS categoria_createdAt, c.updatedAt AS categoria_updatedAt,
  -- Valor final calculado
  CASE
    WHEN p.tempo_experiencia = 3 THEN ROUND(s.vlr_servico * 1.30, 2)
    WHEN p.tempo_experiencia > 3 AND p.tempo_experiencia <= 5 THEN
ROUND(s.vlr_servico * 1.50, 2)
    WHEN p.tempo_experiencia > 5 THEN ROUND(s.vlr_servico * 1.75, 2)
    ELSE s.vlr_servico
  END AS vlr_final,
  CASE
    WHEN p.tempo_experiencia = 3 THEN '30%'
    WHEN p.tempo_experiencia > 3 AND p.tempo_experiencia <= 5 THEN '50%'
    WHEN p.tempo_experiencia > 5 THEN '75%'
    ELSE '0%'
  END AS percentual_acrescimo
FROM servicos s

```

```

INNER JOIN prestadores p ON s.codigo_prestador = p.codigo_prestador
INNER JOIN categorias c ON p.id_categoria = c.id_categoria
ORDER BY s.id_servico;

-- Adicionar serviço
INSERT INTO servicos (nome_servico, codigo_prestador)
VALUES ('{NOME DO SERVIÇO}', {CODIGO DO PRESTADOR});

-- Buscar serviço por ID
SELECT
  s.id_servico, s.nome_servico, s.vlr_servico, s.codigo_prestador,
  s.createdAt, s.updatedAt, p.codigo_prestador AS prestador_codigo,
  p.nome_prestador, p.tempo_experiencia, p.id_categoria,
  p.createdAt AS prestador_createdAt, p.updatedAt AS prestador_updatedAt,
  c.id_categoria, c.nome_categoria, c.createdAt AS categoria_createdAt,
  c.updatedAt AS categoria_updatedAt,
  CASE
    WHEN p.tempo_experiencia = 3 THEN ROUND(s.vlr_servico * 1.30, 2)
    WHEN p.tempo_experiencia > 3 AND p.tempo_experiencia <= 5 THEN
ROUND(s.vlr_servico * 1.50, 2)
    WHEN p.tempo_experiencia > 5 THEN ROUND(s.vlr_servico * 1.75, 2)
    ELSE s.vlr_servico
  END AS vlr_final
FROM servicos s
INNER JOIN prestadores p ON s.codigo_prestador = p.codigo_prestador
INNER JOIN categorias c ON p.id_categoria = c.id_categoria
WHERE s.id_servico = {ID DO SERVIÇO};

-- Buscar serviços por prestador
SELECT
  s.id_servico, s.nome_servico, s.vlr_servico, s.codigo_prestador,
  s.createdAt, s.updatedAt, p.codigo_prestador AS prestador_codigo,
  p.nome_prestador, p.tempo_experiencia, p.id_categoria,
  c.id_categoria, c.nome_categoria,
  CASE
    WHEN p.tempo_experiencia = 3 THEN ROUND(s.vlr_servico * 1.30, 2)
    WHEN p.tempo_experiencia > 3 AND p.tempo_experiencia <= 5 THEN
ROUND(s.vlr_servico * 1.50, 2)
    WHEN p.tempo_experiencia > 5 THEN ROUND(s.vlr_servico * 1.75, 2)
    ELSE s.vlr_servico
  END AS vlr_final
FROM servicos s
INNER JOIN prestadores p ON s.codigo_prestador = p.codigo_prestador
INNER JOIN categorias c ON p.id_categoria = c.id_categoria
WHERE s.codigo_prestador = {CODIGO DO PRESTADOR}
ORDER BY s.nome_servico;

-- Atualizar serviço

```

```

UPDATE servicos SET nome_servico = '{NOVO NOME}',
    codigo_prestador = {NOVO PRESTADOR},
    vlr_servico = {NOVO VALOR},
WHERE id_servico = {ID DO SERVIÇO};
-- Deleta serviço
DELETE FROM servicos WHERE id_servico = {ID DO SERVIÇO};

```

Atividades da N3 de Banco de Dados:

1. Consultas complexas

a. Três tabelas – Objetivo: Listar todos os prestadores com seus serviços e categoria

```

SELECT
    p.codigo_prestador, p.nome_prestador, p.tempo_experiencia,
    c.nome_categoria, s.nome_servico, s.vlr_servico
FROM prestadores p
INNER JOIN categorias c ON p.id_categoria = c.id_categoria
INNER JOIN servicos s ON p.codigo_prestador = s.codigo_prestador
ORDER BY p.nome_prestador, s.nome_servico;

```

```

MariaDB [prestadores_db]> SELECT
->  p.codigo_prestador,
->  p.nome_prestador,
->  p.tempo_experiencia,
->  c.nome_categoria,
->  s.nome_servico,
->  s.vlr_servico
-> FROM prestadores p
-> INNER JOIN categorias c ON p.id_categoria = c.id_categoria
-> INNER JOIN servicos s ON p.codigo_prestador = s.codigo_prestador
-> ORDER BY p.nome_prestador, s.nome_servico;

```

codigo_prestador	nome_prestador	tempo_experiencia	nome_categoria	nome_servico	vlr_servico
5	Ana Costa	5	Eletricista	Instalacao de Ar Condicionado	80.00
11	Antonio Dias	3	Pedreiro	Assentamento de Piso	80.00
11	Antonio Dias	3	Pedreiro	Construcao de Muro	80.00
6	Carlos Eduardo	8	Eletricista	Projeto Eletrico Industrial	80.00
9	Fernando Souza	4	Pintor	Pintura Comercial	80.00
9	Fernando Souza	4	Pintor	Pintura Residencial	80.00
1	Joao Silva	3	Carpintaria edit	Fabricacao de M?vel Planejado	80.00
1	Joao Silva	3	Carpintaria edit	Instalacao de Porta	80.00
1	Joao Silva	3	Carpintaria edit	Teste	80.00
7	Jose Oliveira	3	Encanador	Instalacao de Sistema Hidraulico	80.00
7	Jose Oliveira	3	Encanador	Reparo de Vazamento	80.00
3	Lucas Martins	6	Carpintaria edit	Construcao de Deck	80.00
3	Lucas Martins	6	Carpintaria edit	teste	80.00
12	Marcos Pereira	6	Pedreiro	Reforma Estrutural	80.00
4	Maria Santos	4	Eletricista	Instalacao Eletrica Residencial	80.00
4	Maria Santos	4	Eletricista	Manutencao de Quadro Eletrico	80.00
2	Pedro Alves	2	Carpintaria edit	Reparo em Janela	80.00
10	Ricardo Gomes	10	Pintor	Textura e Grafiato	80.00
8	Roberto Lima	7	Encanador	Desentupimento	80.00
8	Roberto Lima	7	Encanador	Instalacao de Aquecedor	80.00

20 rows in set (0.001 sec)

b. Duas tabelas – Objetivo: Listar a quantidade de prestadores, média de experiência dos prestadores e total de serviços de cada categoria

```
SELECT
  c.nome_categoria,
  COUNT(DISTINCT p.codigo_prestador) as total_prestadores,
  ROUND(AVG(p.tempo_experiencia), 2) as media_experiencia,
  COUNT(s.id_servico) as total_servicos
FROM categorias c
INNER JOIN prestadores p ON c.id_categoria = p.id_categoria
INNER JOIN servicos s ON p.codigo_prestador = s.codigo_prestador
GROUP BY c.id_categoria, c.nome_categoria
HAVING COUNT(DISTINCT p.codigo_prestador) > 0
ORDER BY total_prestadores DESC;
```

```
MariaDB [prestadores_db]> SELECT
->   c.nome_categoria,
->   COUNT(DISTINCT p.codigo_prestador) as total_prestadores,
->   ROUND(AVG(p.tempo_experiencia), 2) as media_experiencia,
->   COUNT(s.id_servico) as total_servicos
-> FROM categorias c
-> INNER JOIN prestadores p ON c.id_categoria = p.id_categoria
-> INNER JOIN servicos s ON p.codigo_prestador = s.codigo_prestador
-> GROUP BY c.id_categoria, c.nome_categoria
-> HAVING COUNT(DISTINCT p.codigo_prestador) > 0
-> ORDER BY total_prestadores DESC;
```

nome_categoria	total_prestadores	media_experiencia	total_servicos
Eletricista	3	5.25	4
Carpintaria edit	3	3.83	6
Encanador	2	5.00	4
Pedreiro	2	4.00	3
Pintor	2	6.00	3

```
5 rows in set (0.001 sec)
```

c. Duas tabelas – Objetivo: Listar prestadores de uma categoria específica (com base no id da categoria)

```
SELECT
  c.id_categoria, c.nome_categoria, p.codigo_prestador,
  p.nome_prestador, p.tempo_experiencia
FROM categorias c
INNER JOIN prestadores p ON c.id_categoria = p.id_categoria
WHERE c.id_categoria = 4;
```

```
MariaDB [prestadores_db]> SELECT
->   c.id_categoria,
->   c.nome_categoria,
->   p.codigo_prestador,
->   p.nome_prestador,
->   p.tempo_experiencia
-> FROM categorias c
-> INNER JOIN prestadores p ON c.id_categoria = p.id_categoria
-> WHERE c.id_categoria = 4;
+-----+-----+-----+-----+-----+
| id_categoria | nome_categoria | codigo_prestador | nome_prestador | tempo_experiencia |
+-----+-----+-----+-----+-----+
|          4 | Pintor        |          9 | Fernando Souza |          4 |
|          4 | Pintor        |         10 | Ricardo Gomes  |         10 |
+-----+-----+-----+-----+-----+
2 rows in set (0.000 sec)
```

2. Views

a. View para métricas e verificar potenciais receitas

```
CREATE VIEW metricas AS
SELECT
  c.id_categoria, c.nome_categoria,
  COUNT(DISTINCT p.codigo_prestador) as qtd_prestadores,
  COUNT(s.id_servico) as qtd_servicos,
  ROUND(AVG(p.tempo_experiencia), 2) as experiencia_media,
  ROUND(SUM(s.vlr_servico), 2) as receita_potencial_total
FROM categorias c
LEFT JOIN prestadores p ON c.id_categoria = p.id_categoria
LEFT JOIN servicos s ON p.codigo_prestador = s.codigo_prestador
GROUP BY c.id_categoria, c.nome_categoria, c.createdAt;
```

```
MariaDB [prestadores_db]> CREATE VIEW metricas AS
-> SELECT
->   c.id_categoria,
->   c.nome_categoria,
->   COUNT(DISTINCT p.codigo_prestador) as qtd_prestadores,
->   COUNT(s.id_servico) as qtd_servicos,
->   ROUND(AVG(p.tempo_experiencia), 2) as experiencia_media,
->   ROUND(SUM(s.vlr_servico), 2) as receita_potencial_total
-> FROM categorias c
-> LEFT JOIN prestadores p ON c.id_categoria = p.id_categoria
-> LEFT JOIN servicos s ON p.codigo_prestador = s.codigo_prestador
-> GROUP BY c.id_categoria, c.nome_categoria, c.createdAt;
Query OK, 0 rows affected (0.006 sec)
```

```
MariaDB [prestadores_db]> SELECT * from metricas;
```

id_categoria	nome_categoria	qtd_prestadores	qtd_servicos	experiencia_media	receita_potencial_total
1	Carpintaria edit	3	6	3.83	480.00
2	Eletricista	3	4	5.25	320.00
3	Encanador	2	4	5.00	320.00
4	Pintor	2	3	6.00	240.00
5	Pedreiro	2	3	4.00	240.00

5 rows in set (0.001 sec)

b. View para verificar serviços ofertados por cada prestador

```
CREATE VIEW servicos_oferecidos_por_prestador AS
SELECT
    p.codigo_prestador,
    p.nome_prestador,
    p.tempo_experiencia,
    c.nome_categoria,
    COUNT(s.id_servico) as total_servicos,
    GROUP_CONCAT(s.nome_servico ORDER BY s.nome_servico SEPARATOR ', ') as
servicos_oferecidos
FROM prestadores p
INNER JOIN categorias c ON p.id_categoria = c.id_categoria
LEFT JOIN servicos s ON p.codigo_prestador = s.codigo_prestador
GROUP BY
    p.codigo_prestador,
    p.nome_prestador,
    p.tempo_experiencia,
    c.nome_categoria;
```

```
MariaDB [prestadores_db]> CREATE VIEW servicos_oferecidos_por_prestador AS
-> SELECT
->     p.codigo_prestador,
->     p.nome_prestador,
->     p.tempo_experiencia,
->     c.nome_categoria,
->     COUNT(s.id_servico) as total_servicos,
->     GROUP_CONCAT(s.nome_servico ORDER BY s.nome_servico SEPARATOR ', ') as servicos_oferecidos
-> FROM prestadores p
-> INNER JOIN categorias c ON p.id_categoria = c.id_categoria
-> LEFT JOIN servicos s ON p.codigo_prestador = s.codigo_prestador
-> GROUP BY
->     p.codigo_prestador,
->     p.nome_prestador,
->     p.tempo_experiencia,
->     c.nome_categoria
-> ;
Query OK, 0 rows affected (0.002 sec)
```

```
MariaDB [prestadores_db]> select * from servicos_oferecidos_por_prestador;
```

codigo_prestador	nome_prestador	tempo_experiencia	nome_categoria	total_servicos	servicos_oferecidos
1	Joao Silva	3	Carpintaria edit	3	Fabricacao de M?vel Planejado, Instalacao de Porta, Teste
2	Pedro Alves	2	Carpintaria edit	1	Reparo em Janela
3	Lucas Martins	6	Carpintaria edit	2	Construcao de Deck, teste
4	Maria Santos	4	Eletricista	2	Instalacao Eletrica Residencial, Manutencao de Quadro Eletrico
5	Ana Costa	5	Eletricista	1	Instalacao de Ar Condicionado
6	Carlos Eduardo	8	Eletricista	1	Projeto Eletrico Industrial
7	Jose Oliveira	3	Encanador	2	Instalacao de Sistema Hidraulico, Reparo de Vazamento
8	Roberto Lima	7	Encanador	2	Desentupimento, Instalacao de Aquecedor
9	Fernando Souza	4	Pintor	2	Pintura Comercial, Pintura Residencial
10	Ricardo Gomes	10	Pintor	1	Textura e Grafiato
11	Antonio Dias	3	Pedreiro	2	Assentamento de Piso, Construcao de Muro
12	Marcos Pereira	6	Pedreiro	1	Reforma Estrutural

```
12 rows in set (0.004 sec)
```

3. Trigger (Atualizar o updatedAt do prestador após inserir um serviço para ele)

```
DELIMITER $$
CREATE TRIGGER trg_atualiza_prestador_updatedAt
AFTER INSERT ON servicos
FOR EACH ROW
BEGIN
    UPDATE prestadores
    SET updatedAt = CURRENT_TIMESTAMP
    WHERE codigo_prestador = NEW.codigo_prestador;
END$$
```

```
MariaDB [prestadores_db]> select * from prestadores;
```

codigo_prestador	nome_prestador	tempo_experiencia	id_categoria	createdAt	updatedAt
1	Joao Silva	3	1	0000-00-00 00:00:00	0000-00-00 00:00:00
2	Pedro Alves	2	1	0000-00-00 00:00:00	0000-00-00 00:00:00
3	Lucas Martins	6	1	0000-00-00 00:00:00	0000-00-00 00:00:00
4	Maria Santos	4	2	0000-00-00 00:00:00	0000-00-00 00:00:00
5	Ana Costa	5	2	0000-00-00 00:00:00	0000-00-00 00:00:00
6	Carlos Eduardo	8	2	0000-00-00 00:00:00	0000-00-00 00:00:00
7	Jose Oliveira	3	3	0000-00-00 00:00:00	0000-00-00 00:00:00
8	Roberto Lima	7	3	0000-00-00 00:00:00	0000-00-00 00:00:00
9	Fernando Souza	4	4	0000-00-00 00:00:00	0000-00-00 00:00:00
10	Ricardo Gomes	10	4	0000-00-00 00:00:00	0000-00-00 00:00:00
11	Antonio Dias	3	5	0000-00-00 00:00:00	0000-00-00 00:00:00
12	Marcos Pereira	6	5	0000-00-00 00:00:00	0000-00-00 00:00:00

12 rows in set (0.000 sec)

```
MariaDB [prestadores_db]> INSERT INTO servicos (nome_servico, codigo_prestador) VALUES ('Teste', 12);
Query OK, 1 row affected, 2 warnings (0.004 sec)
```

```
MariaDB [prestadores_db]> select * from prestadores;
```

codigo_prestador	nome_prestador	tempo_experiencia	id_categoria	createdAt	updatedAt
1	Joao Silva	3	1	0000-00-00 00:00:00	0000-00-00 00:00:00
2	Pedro Alves	2	1	0000-00-00 00:00:00	0000-00-00 00:00:00
3	Lucas Martins	6	1	0000-00-00 00:00:00	0000-00-00 00:00:00
4	Maria Santos	4	2	0000-00-00 00:00:00	0000-00-00 00:00:00
5	Ana Costa	5	2	0000-00-00 00:00:00	0000-00-00 00:00:00
6	Carlos Eduardo	8	2	0000-00-00 00:00:00	0000-00-00 00:00:00
7	Jose Oliveira	3	3	0000-00-00 00:00:00	0000-00-00 00:00:00
8	Roberto Lima	7	3	0000-00-00 00:00:00	0000-00-00 00:00:00
9	Fernando Souza	4	4	0000-00-00 00:00:00	0000-00-00 00:00:00
10	Ricardo Gomes	10	4	0000-00-00 00:00:00	0000-00-00 00:00:00
11	Antonio Dias	3	5	0000-00-00 00:00:00	0000-00-00 00:00:00
12	Marcos Pereira	6	5	0000-00-00 00:00:00	2025-12-08 15:28:52

12 rows in set (0.000 sec)

4. Stored Procedure (Reajustar todos os serviços de um prestador específico e retorna os serviços que tiveram valores alterados)

```
CREATE PROCEDURE sp_reajustar_servicos_prestador(  
    IN p_codigo_prestador INT,  
    IN p_percentual_reajuste DECIMAL(5,2)  
)  
BEGIN  
    UPDATE servicos  
    SET vlr_servico = vlr_servico * (1 + p_percentual_reajuste / 100),  
        updatedAt = CURRENT_TIMESTAMP  
    WHERE codigo_prestador = p_codigo_prestador;  
    SELECT  
        s.id_servico,  
        s.nome_servico,  
        s.vlr_servico as novo_valor,  
        p.nome_prestador  
    FROM servicos s  
    INNER JOIN prestadores p ON s.codigo_prestador = p.codigo_prestador  
    WHERE s.codigo_prestador = p_codigo_prestador;  
END$  
DELIMITER ;
```

```

MariaDB [prestadores_db]> SELECT
  -> s.id_servico,
  -> s.nome_servico,
  -> s.vlr_servico,
  -> p.nome_prestador
  -> FROM servicos s
  -> INNER JOIN prestadores p on s.codigo_prestador = p.codigo_prestador
  -> WHERE s.codigo_prestador = 3;

```

id_servico	nome_servico	vlr_servico	nome_prestador
4	Construcao de Deck	80.00	Lucas Martins
20	teste	80.00	Lucas Martins

2 rows in set (0.000 sec)

```

MariaDB [prestadores_db]> CALL sp_reajustar_servicos_prestador (3, 10);

```

id_servico	nome_servico	novo_valor	nome_prestador
4	Construcao de Deck	88.00	Lucas Martins
20	teste	88.00	Lucas Martins

2 rows in set (0.003 sec)

Query OK, 3 rows affected (0.007 sec)

```

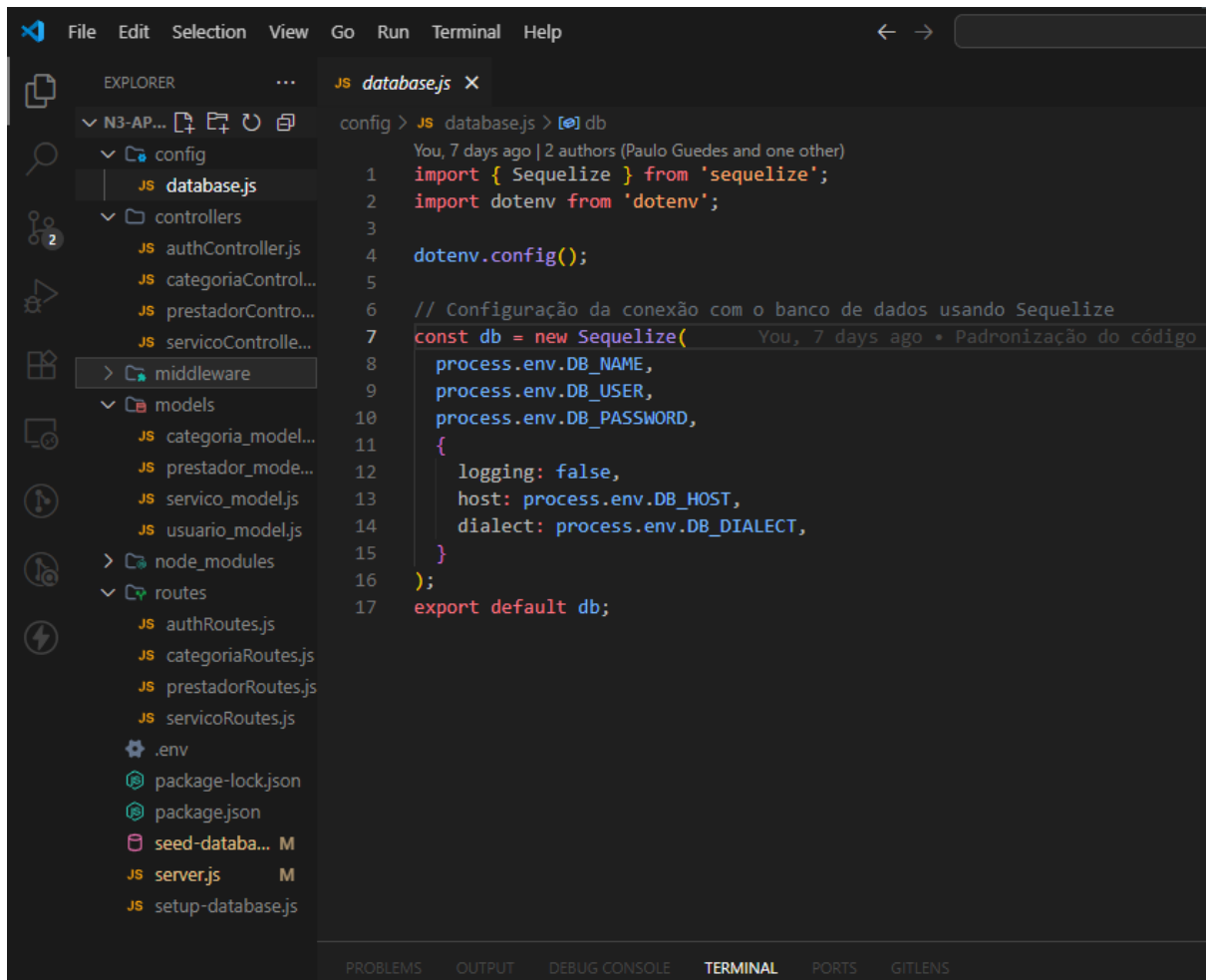
MariaDB [prestadores_db]> SELECT
  -> s.id_servico,
  -> s.nome_servico,
  -> s.vlr_servico,
  -> p.nome_prestador
  -> FROM servicos s
  -> INNER JOIN prestadores p on s.codigo_prestador = p.codigo_prestador
  -> WHERE s.codigo_prestador = 3;

```

id_servico	nome_servico	vlr_servico	nome_prestador
4	Construcao de Deck	88.00	Lucas Martins
20	teste	88.00	Lucas Martins

2 rows in set (0.000 sec)

CÓDIGOS FONTE UTILIZADOS NA API ORM



```
File Edit Selection View Go Run Terminal Help

EXPLORER
N3-AP...
  config
  database.js
  controllers
    authController.js
    categoriaControl...
    prestadorContro...
    servicoControlle...
  middleware
  models
    categoria_model...
    prestador_mode...
    servico_model.js
    usuario_model.js
  node_modules
  routes
    authRoutes.js
    categoriaRoutes.js
    prestadorRoutes.js
    servicoRoutes.js
  .env
  package-lock.json
  package.json
  seed-databa... M
  server.js M
  setup-database.js

JS database.js
config > JS database.js > db
You, 7 days ago | 2 authors (Paulo Guedes and one other)
1 import { Sequelize } from 'sequelize';
2 import dotenv from 'dotenv';
3
4 dotenv.config();
5
6 // Configuração da conexão com o banco de dados usando Sequelize
7 const db = new Sequelize(
8   process.env.DB_NAME,
9   process.env.DB_USER,
10  process.env.DB_PASSWORD,
11  {
12    logging: false,
13    host: process.env.DB_HOST,
14    dialect: process.env.DB_DIALECT,
15  }
16 );
17 export default db;
```


JS categoriaRoutes.js X

routes > JS categoriaRoutes.js > [🔗] default

You, 7 days ago | 2 authors (Paulo Guedes and one other)

```
1 import { Router } from 'express';
2 import {
3   criarCategoria,
4   listarCategorias,
5   buscarCategoriaPorId,
6   atualizarCategoria,
7   deletarCategoria
8 } from '../controllers/categoriaController.js';
9 import verificarToken from '../middleware/auth.js';
10
11 const router = Router();
12
13 router.post('/', verificarToken, criarCategoria);
14 router.get('/', listarCategorias);
15 router.get('/:id', buscarCategoriaPorId);
16 router.put('/:id', verificarToken, atualizarCategoria);
17 router.delete('/:id', verificarToken, deletarCategoria);
18
19 export default router;
```

Paulo Guedes, 2 weeks ago • Pr

JS prestadorRoutes.js X

routes > JS prestadorRoutes.js > ...

You, 7 days ago | 2 authors (Paulo Guedes and one other)

```
1 import { Router } from 'express';
2 import {
3   criarPrestador,
4   listarPrestadores,
5   buscarPrestadorPorId,
6   buscarPrestadoresPorCategoria,
7   buscarPrestadoresPorServico,
8   atualizarPrestador,
9   deletarPrestador
10 } from '../controllers/prestadorController.js';
11 import verificarToken from '../middleware/auth.js';
12
13 const router = Router();
14
15 router.post('/', verificarToken, criarPrestador);
16 router.get('/', listarPrestadores);
17 router.get('/categoria/:id_categoria', buscarPrestadoresPorCategoria);
18 router.get('/servico/:id_servico', buscarPrestadoresPorServico);
19 router.get('/:id', buscarPrestadorPorId);
20 router.put('/:id', verificarToken, atualizarPrestador);
21 router.delete('/:id', verificarToken, deletarPrestador);
22
23 export default router;
```

JS `servicoRoutes.js` X

routes > JS `servicoRoutes.js` > [🔍] default

You, 7 days ago | 2 authors (Paulo Guedes and one other)

```
1  ∨ import { Router } from 'express';
2  ∨ import {
3      criarServico,
4      listarServicos,
5      buscarServicoPorId,
6      buscarServicosPorPrestador,
7      atualizarServico,
8      deletarServico
9  } from '../controllers/servicoController.js';
10 import verificarToken from '../middleware/auth.js';
11
12 const router = Router();
13
14 router.post('/', verificarToken, criarServico);
15 router.get('/', listarServicos);
16 router.get('/prestador/:codigo_prestador', buscarServicosPorPrestador);
17 router.get('/:id', buscarServicoPorId);
18 router.put('/:id', verificarToken, atualizarServico);
19 router.delete('/:id', verificarToken, deletarServico);
20
21 export default router;
```

Paulo Guedes, 2 weeks ago • Primeiro commit

CONTROLLERS:

//Categoria

```
import Categoria from "../models/categoria_model.js";
import Prestador from "../models/prestador_model.js";

// CREATE - Criar categoria
export const criarCategoria = async (req, res) => {
  try {
    const { nome_categoria } = req.body;
    if (!nome_categoria) {
      return res.status(400).json({ message: "O campo 'nome_categoria' é obrigatório." });
    }

    const categoria = await Categoria.create({ nome_categoria });

    res.status(201).json({
      message: "Categoria criada com sucesso",
      categoria,
    });
  } catch (error) {
    res.status(500).json({
```

```

        message: "Erro ao criar categoria",
        error: error.message,
    });
}
};

// READ - Listar todas categorias
export const listarCategorias = async (req, res) => {
    try {
        const categorias = await Categoria.findAll({
            include: [
                {
                    model: Prestador,
                    as: "prestadores",
                    attributes: ["codigo_prestador", "nome_prestador",
"tempo_experiencia"],
                },
            ],
        });

        res.json(categorias);
    } catch (error) {
        res.status(500).json({
            message: "Erro ao listar categorias",
            error: error.message,
        });
    }
};

// READ - Buscar categoria por ID
export const buscarCategoriaPorId = async (req, res) => {
    try {
        const { id } = req.params;

        const categoria = await Categoria.findByPk(id, {
            include: [
                {
                    model: Prestador,
                    as: "prestadores",
                },
            ],
        });

        if (!categoria) {
            return res.status(404).json({ message: "Categoria não encontrada" });
        }
    }
};

```

```

    res.json(categoria);
  } catch (error) {
    res.status(500).json({
      message: "Erro ao buscar categoria",
      error: error.message,
    });
  }
};

// UPDATE - Atualizar categoria
export const atualizarCategoria = async (req, res) => {
  try {
    const { id } = req.params;
    const { nome_categoria } = req.body;

    const categoria = await Categoria.findByPk(id);

    if (!categoria) {
      return res.status(404).json({ message: "Categoria não encontrada" });
    }

    await categoria.update({ nome_categoria });

    res.json({
      message: "Categoria atualizada com sucesso",
      categoria,
    });
  } catch (error) {
    res.status(500).json({
      message: "Erro ao atualizar categoria",
      error: error.message,
    });
  }
};

// DELETE - Deletar categoria
export const deletarCategoria = async (req, res) => {
  try {
    const { id } = req.params;

    const categoria = await Categoria.findByPk(id);

    if (!categoria) {
      return res.status(404).json({ message: "Categoria não encontrada" });
    }
  }
};

```

```

    await categoria.destroy();

    res.json({ message: "Categoria deletada com sucesso" });
  } catch (error) {
    // Erro de prestadores vinculados à categoria
    if (
      error.name === "SequelizeForeignKeyConstraintError"
      || error.original?.errno === 1451 ||
      error.parent?.errno === 1451
    ) {
      return res.status(400).json({
        message: "Não é possível deletar a categoria pois existem prestadores vinculados a ela.",
      });
    }
    res.status(500).json({
      message: "Erro ao deletar categoria",
      error: error.message,
    });
  }
};

```

//Prestador

```

import Prestador from "../models/prestador_model.js";
import Categoria from "../models/categoria_model.js";
import Servico from "../models/servico_model.js";

// CREATE - Criar prestador
export const criarPrestador = async (req, res) => {
  try {
    const { nome_prestador, tempo_experiencia, id_categoria } = req.body;
    if (!nome_prestador || !tempo_experiencia || !id_categoria) {
      return res.status(400).json({ message: "nome_prestador, tempo_experiencia e id_categoria são obrigatórios" });
    }
    // Verifica se a categoria existe
    const categoria = await Categoria.findByPk(id_categoria);
    if (!categoria) {
      return res.status(404).json({ message: "Categoria não encontrada" });
    }

    const prestador = await Prestador.create({
      nome_prestador,

```

```

        tempo_experiencia,
        id_categoria,
    });

    // Retorna prestador com categoria
    const prestadorCompleto = await
Prestador.findByPk(prestador.codigo_prestador, {
    include: [{ model: Categoria, as: "categoria" }],
});

    res.status(201).json({
        message: "Prestador criado com sucesso",
        prestador: prestadorCompleto,
    });
} catch (error) {
    res.status(500).json({
        message: "Erro ao criar prestador",
        error: error.message,
    });
}
};

// READ - Listar todos prestadores
export const listarPrestadores = async (req, res) => {
    try {
        const prestadores = await Prestador.findAll({
            include: [
                {
                    model: Categoria,
                    as: "categoria",
                    attributes: ["id_categoria", "nome_categoria"],
                },
                {
                    model: Servico,
                    as: "servicos",
                    attributes: ["id_servico", "nome_servico", "vlr_servico"],
                },
            ],
        });

        res.json(prestadores);
    } catch (error) {
        res.status(500).json({
            message: "Erro ao listar prestadores",
            error: error.message,
        });
    }
};

```

```

    }
  };

  // READ - Buscar prestador por ID
  export const buscarPrestadorPorId = async (req, res) => {
    try {
      const { id } = req.params;

      const prestador = await Prestador.findByPk(id, {
        include: [
          { model: Categoria, as: "categoria" },
          { model: Servico, as: "servicos" },
        ],
      });

      if (!prestador) {
        return res.status(404).json({ message: "Prestador não encontrado" });
      }

      res.json(prestador);
    } catch (error) {
      res.status(500).json({
        message: "Erro ao buscar prestador",
        error: error.message,
      });
    }
  };

  // READ - Buscar prestadores por categoria (REQUISITO DO TRABALHO)
  export const buscarPrestadoresPorCategoria = async (req, res) => {
    try {
      const { id_categoria } = req.params;

      const prestadores = await Prestador.findAll({
        where: { id_categoria },
        include: [
          { model: Categoria, as: "categoria" },
          { model: Servico, as: "servicos" },
        ],
      });

      if (prestadores.length === 0) {
        return res.status(404).json({
          message: "Nenhum prestador encontrado para esta categoria",
        });
      }
    }
  };

```



```

    res.json(prestadores);
  } catch (error) {
    res.status(500).json({
      message: "Erro ao buscar prestadores por categoria",
      error: error.message,
    });
  }
};

// READ - Buscar prestadores por serviço (REQUISITO DO TRABALHO)
export const buscarPrestadoresPorServico = async (req, res) => {
  try {
    const { id_servico } = req.params;

    const prestadores = await Prestador.findAll({
      include: [
        {
          model: Servico,
          as: "servicos",
          where: { id_servico },
        },
      ],
    });

    if (prestadores.length === 0) {
      return res.status(404).json({
        message: "Nenhum prestador encontrado para este serviço",
      });
    }

    res.json(prestadores);
  } catch (error) {
    res.status(500).json({
      message: "Erro ao buscar prestadores por serviço",
      error: error.message,
    });
  }
};

// UPDATE - Atualizar prestador
export const atualizarPrestador = async (req, res) => {
  try {
    const { id } = req.params;
    const { nome_prestador, tempo_experiencia, id_categoria } = req.body;

```

```

    const prestador = await Prestador.findByPk(id);

    if (!prestador) {
        return res.status(404).json({ message: "Prestador não encontrado" });
    }

    // Se mudou categoria, verifica se existe
    if (id_categoria && id_categoria !== prestador.id_categoria) {
        const categoria = await Categoria.findByPk(id_categoria);
        if (!categoria) {
            return res.status(404).json({ message: "Categoria não encontrada"
        });
        }
    }

    await prestador.update({ nome_prestador, tempo_experiencia, id_categoria
    });

    const prestadorAtualizado = await Prestador.findByPk(id, {
        include: [{ model: Categoria, as: "categoria" }],
    });

    res.json({
        message: "Prestador atualizado com sucesso",
        prestador: prestadorAtualizado,
    });
} catch (error) {
    res.status(500).json({
        message: "Erro ao atualizar prestador",
        error: error.message,
    });
}
};

// DELETE - Deletar prestador
export const deletarPrestador = async (req, res) => {
    try {
        const { id } = req.params;

        const prestador = await Prestador.findByPk(id);

        if (!prestador) {
            return res.status(404).json({ message: "Prestador não encontrado" });
        }

        await prestador.destroy();
    }
};

```

```

    res.json({ message: "Prestador deletado com sucesso" });
  } catch (error) {
    // Erro de serviços vinculados ao prestador
    if (
      error.name === "SequelizeForeignKeyConstraintError"
      || error.original?.errno === 1451 ||
      error.parent?.errno === 1451
    ) {
      return res.status(400).json({
        message: "Não é possível deletar o prestador pois existem serviços vinculados a ele.",
      });
    }
    res.status(500).json({
      message: "Erro ao deletar prestador",
      error: error.message,
    });
  }
};

```

//Serviço

```

import Servico from "../models/servico_model.js";
import Prestador from "../models/prestador_model.js";
import Categoria from "../models/categoria_model.js";

// CREATE - Criar serviço
export const criarServico = async (req, res) => {
  try {
    const { nome_servico, codigo_prestador } = req.body;
    if (!nome_servico || !codigo_prestador) {
      return res.status(400).json({ message: "Os campos 'nome_servico' e 'codigo_prestador' são obrigatórios." });
    }

    // Verifica se o prestador existe
    const prestador = await Prestador.findByPk(codigo_prestador);
    if (!prestador) {
      return res.status(404).json({ message: "Prestador não encontrado" });
    }

    const servico = await Servico.create({
      nome_servico,
      codigo_prestador,
    });
  }
};

```

```

// Retorna serviço com prestador para calcular vlr_final
const servicoCompleto = await Servico.findByPk(servico.id_servico, {
  include: [
    {
      model: Prestador,
      as: "prestador",
      include: [{ model: Categoria, as: "categoria" }],
    },
  ],
});

res.status(201).json({
  message: "Serviço criado com sucesso",
  servico: {
    ...servicoCompleto.toJSON(),
    vlr_final: servicoCompleto.vlr_final,
  },
});
} catch (error) {
  res.status(500).json({
    message: "Erro ao criar serviço",
    error: error.message,
  });
}
};

// READ - Listar todos serviços
export const listarServicos = async (req, res) => {
  try {
    const servicos = await Servico.findAll({
      include: [
        {
          model: Prestador,
          as: "prestador",
          include: [{ model: Categoria, as: "categoria" }],
        },
      ],
    });

    // Adiciona vlr_final calculado em cada serviço
    const servicosComValorFinal = servicos.map((servico) => ({
      ...servico.toJSON(),
      vlr_final: servico.vlr_final,
    }));
  }
};

```

```

    res.json(servicosComValorFinal);
  } catch (error) {
    res.status(500).json({
      message: "Erro ao listar serviços",
      error: error.message,
    });
  }
};

// READ - Buscar serviço por ID
export const buscarServicoPorId = async (req, res) => {
  try {
    const { id } = req.params;

    const servico = await Servico.findByPk(id, {
      include: [
        {
          model: Prestador,
          as: "prestador",
          include: [{ model: Categoria, as: "categoria" }],
        },
      ],
    });

    if (!servico) {
      return res.status(404).json({ message: "Serviço não encontrado" });
    }

    res.json({
      ...servico.toJSON(),
      vlr_final: servico.vlr_final,
    });
  } catch (error) {
    res.status(500).json({
      message: "Erro ao buscar serviço",
      error: error.message,
    });
  }
};

// READ - Buscar serviços por prestador (REQUISITO DO TRABALHO)
export const buscarServicosPorPrestador = async (req, res) => {
  try {
    const { codigo_prestador } = req.params;

    const servicos = await Servico.findAll({

```

```

    where: { codigo_prestador },
    include: [
      {
        model: Prestador,
        as: "prestador",
        include: [{ model: Categoria, as: "categoria" }],
      },
    ],
  });

  if (servicos.length === 0) {
    return res.status(404).json({
      message: "Nenhum serviço encontrado para este prestador",
    });
  }

  const servicosComValorFinal = servicos.map((servico) => ({
    ...servico.toJSON(),
    vlr_final: servico.vlr_final,
  }));

  res.json(servicosComValorFinal);
} catch (error) {
  res.status(500).json({
    message: "Erro ao buscar serviços por prestador",
    error: error.message,
  });
}
};

// UPDATE - Atualizar serviço
export const atualizarServico = async (req, res) => {
  try {
    const { id } = req.params;
    const { nome_servico, codigo_prestador, vlr_servico } = req.body;

    const servico = await Servico.findByPk(id);

    if (!servico) {
      return res.status(404).json({ message: "Serviço não encontrado" });
    }

    // Se mudou prestador, verifica se existe
    if (codigo_prestador && codigo_prestador !== servico.codigo_prestador) {
      const prestador = await Prestador.findByPk(codigo_prestador);
      if (!prestador) {

```

```

        return res.status(404).json({ message: "Prestador não encontrado"
    });
    }
}

await servico.update({ nome_servico, codigo_prestador, vlr_servico });

const servicoAtualizado = await Servico.findByPk(id, {
  include: [
    {
      model: Prestador,
      as: "prestador",
      include: [{ model: Categoria, as: "categoria" }],
    },
  ],
});

res.json({
  message: "Serviço atualizado com sucesso",
  servico: {
    ...servicoAtualizado.toJSON(),
    vlr_final: servicoAtualizado.vlr_final,
  },
});
} catch (error) {
  res.status(500).json({
    message: "Erro ao atualizar serviço",
    error: error.message,
  });
}
};

// DELETE - Deletar serviço
export const deletarServico = async (req, res) => {
  try {
    const { id } = req.params;

    const servico = await Servico.findByPk(id);

    if (!servico) {
      return res.status(404).json({ message: "Serviço não encontrado" });
    }

    await servico.destroy();

    res.json({ message: "Serviço deletado com sucesso" });
  }
};

```

```

    } catch (error) {
      res.status(500).json({
        message: "Erro ao deletar serviço",
        error: error.message,
      });
    }
  });
};

```

MODELS:

```

//Categoria
import { DataTypes } from 'sequelize';
import db from '../config/database.js';

// Modelo da Categoria (carpintaria, eletricista, encanador, etc)
const Categoria = db.define('Categoria', {
  id_categoria: {
    type: DataTypes.INTEGER,
    primaryKey: true,
    autoIncrement: true
  },
  nome_categoria: {
    type: DataTypes.STRING(100),
    allowNull: false,
    validate: {
      notEmpty: {
        msg: 'Nome da categoria não pode ser vazio'
      }
    }
  }
}, {
  tableName: 'categorias',
});

export default Categoria;

```

```

//Prestador
import { DataTypes } from 'sequelize';
import db from '../config/database.js';
import Categoria from './categoria_model.js';

// Modelo do Prestador de Serviço
const Prestador = db.define('Prestador', {
  codigo_prestador: {
    type: DataTypes.INTEGER,
    primaryKey: true,

```



```

    autoIncrement: true
  },
  nome_prestador: {
    type: DataTypes.STRING(150),
    validate: {
      notEmpty: {
        msg: 'Nome do prestador não pode ser vazio'
      }
    }
  },
  tempo_experiencia: {
    type: DataTypes.INTEGER,
    validate: {
      min: {
        args: [0],
        msg: 'Tempo de experiência deve ser maior ou igual a 0'
      }
    }
  },
  id_categoria: {
    type: DataTypes.INTEGER,
    references: {
      model: 'categorias',
      key: 'id_categoria'
    }
  }
}, {
  tableName: 'prestadores',
  timestamps: true
});

```

// Relacionamento: Prestador pertence a uma Categoria

```

Prestador.belongsTo(Categoria, {
  foreignKey: 'id_categoria',
  as: 'categoria'
});

```

```

Categoria.hasMany(Prestador, {
  foreignKey: 'id_categoria',
  as: 'prestadores'
});

```

```

export default Prestador;

```

//Servico

```

import { DataTypes } from "sequelize";
import db from "../config/database.js";

```

```

import Prestador from "../prestador_model.js";

// Modelo do Serviço
const Servico = db.define(
  "Servico",
  {
    id_servico: {
      type: DataTypes.INTEGER,
      primaryKey: true,
      autoIncrement: true,
    },
    nome_servico: {
      type: DataTypes.STRING(150),
      validate: {
        notEmpty: {
          msg: "Nome do serviço não pode ser vazio",
        },
      },
    },
    vlr_servico: {
      type: DataTypes.DECIMAL(5, 2),
      defaultValue: 80.0,
    },
    codigo_prestador: {
      type: DataTypes.INTEGER,
      references: {
        model: "prestadores",
        key: "codigo_prestador",
      },
    },
    vlr_final: {
      type: DataTypes.VIRTUAL,
      get() {
        const valorBase = parseFloat(this.vlr_servico);
        const prestador = this.prestador;

        if (!prestador) return valorBase;

        const experiencia = prestador.tempo_experiencia;
        let acrescimo = 0;

        if (experiencia === 3) {
          acrescimo = 0.3; // 30%
        } else if (experiencia > 3 && experiencia <= 5) {
          acrescimo = 0.5; // 50%
        } else if (experiencia > 5) {

```

```

        acrescimo = 0.75; // 75%
    }

    return (valorBase + valorBase * acrescimo).toFixed(2);
},
},
},
{
    tableName: "servicos",
    timestamps: true,
}
);

// Relacionamento: Serviço pertence a um Prestador
Servico.belongsTo(Prestador, {
    foreignKey: "codigo_prestador",
    as: "prestador",
});

Prestador.hasMany(Servico, {
    foreignKey: "codigo_prestador",
    as: "servicos",
});

export default Servico;

```